INSTITUTE FOR ADVANCED COMPUTING AND SOFTWARE DEVELOPMENT
AKURDI, PUNE – 411044

Documentation On

"End-to-End Nifty Stock Forecasting Pipeline"

PGCP-BDA FEB 2026

Submitted By:
Group No: 01

Asrar G Shaikh (262504)
Vrushabh K Urkudkar (262555)

Dr. Shantanu Pathak
Project Guide

Mr. Anil Sharma
Centre Coordinator

DECLARATION

We hereby declare that the project report titled "End-to-End Nifty Stock Forecasting Pipeline", submitted by us to the Institute For Advanced Computing And Software Development, Akurdi, Pune, in fulfilment of the requirements for the award of the degree of Post Graduate Diploma in Big Data Analytics (PGCP-BDA), under the guidance of Dr. Shantanu Pathak, is our original work.

We have not copied any code or content from any source without proper attribution, and we have not allowed anyone else to copy our work. The project was completed using Python, Airflow, Azure Data Factory, ADLS Gen2, Databricks, PySpark, LSTM, and Prophet, and was developed as part of our academic coursework.

We further confirm that this project is original and has not been submitted previously for any other academic or professional purpose.

Place: Signature:

Date: Name: Asrar Shaikh / Vrushabh Urkudkar

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to Dr. Shantanu Pathak, Project Guide, for providing us with the guidance and support to complete this academic project. His valuable insights, expertise, and encouragement have been instrumental in the success of this project.

We would also like to thank our fellow classmates for their support and cooperation during the project. Their feedback and suggestions were helpful in improving the quality of the project.

We would like to extend our gratitude to Mr. Anil Sharma, Centre Coordinator, for providing us with the necessary resources and facilities to complete this project. His support has been crucial in the timely completion of this project.

Finally, we would like to thank our families and friends for their constant encouragement and support throughout the project. Their belief in us has been a constant source of motivation and inspiration.

Thank you all for your support and guidance in completing this academic project.

ABSTRACT

This project implements an automated End-to-End Nifty Stock Forecasting Pipeline using Microsoft Azure cloud services, Apache Airflow, Azure Data Factory, Azure Databricks, and PySpark. Historical Nifty stock-market data is fetched from Yahoo Finance using the yfinance library and ingested into the Bronze layer of Azure Data Lake Storage Gen2. The data is then processed using Databricks notebooks, where it is cleaned, validated, and standardized before being stored in the Silver layer.

Feature engineering is performed on the processed data to create an analytics-ready Gold layer. The Gold data is used to train two forecasting models, LSTM and Prophet, to predict next-day stock prices and evaluate their performance. Apache Airflow and Azure Data Factory automate and orchestrate the pipeline, while watermark-based incremental processing ensures that only newly available data is processed instead of reprocessing the complete historical dataset. This architecture provides an automated, scalable, and production-oriented workflow for Nifty stock-market data processing and forecasting.

Table Of Contents

1. INTRODUCTION 1
1.1 Problem Statement 2
1.2 Scope 3
1.3 Aim & Objectives 4
1.4 Business Problem 5
2. PROJECT DESCRIPTION 6
2.1 Project Work Flow 6
2.2 Project Architecture Overview 6
2.3 Data Source & Collection 7
2.4 Dataset Details 8
2.5 Exploratory Data Analysis 8
2.6 EDA Findings & Conclusions 9
3. SYSTEM ARCHITECTURE 10
3.1 Technologies Used & Their Roles 10
3.2 Azure Services & Their Roles 10
3.3 Airflow & Azure Data Factory Orchestration 11
3.4 Azure Databricks & PySpark Processing 11
3.5 Medallion Architecture (Bronze, Silver & Gold) 11
3.6 Advantages, Limitations & Technology Selection 12
4. IMPLEMENTATION DETAILS 13
4.1 Data Preprocessing & Feature Engineering 13
4.2 LSTM Model Training & Forecasting 13
4.3 Prophet Model Training & Forecasting 14
4.4 LSTM vs Prophet Model Comparison 14
4.5 Model Evaluation Metrics 14
4.6 Incremental Processing & Watermark Strategy 14
4.7 Code Implementation & Explanation 15
4.8 Problems Faced & Solutions 15
4.9 Quality Assurance, Testing & Exception Handling 16
5. PROJECT REQUIREMENTS & RESULTS 17
5.1 Hardware Requirements 17
5.2 Software & Libraries Used 17
5.3 Azure Resources & Configuration 17

5.4 Pipeline Deployment & Execution 18
5.5 Forecasting Results 18
5.6 Model Performance Comparison 18
5.7 Project Screenshots 19
6. FUTURE SCOPE 20
7. CONCLUSION 21
8. REFERENCES 22

Table of Figure:

Figure 1 Project work flow diagram 6

CHAPTER 1
INTRODUCTION

The stock market generates a large amount of historical data every trading day, making automated data processing and forecasting important for financial analysis. Stock-market data is time-series data, where historical observations such as Open, High, Low, Close, and Volume can be used to identify patterns and generate forecasts. Manual collection and processing of this data can be time-consuming and difficult to maintain, especially when the pipeline needs to run regularly.

This project presents an automated, cloud-based End-to-End Nifty Stock Forecasting Pipeline that integrates Microsoft Azure services with distributed data processing and machine learning techniques. Historical Nifty stock-market data is collected using the yfinance library and stored in Azure Data Lake Storage Gen2. The data is processed using Azure Databricks and PySpark through a Medallion Architecture consisting of Bronze, Silver, and Gold layers.

The pipeline is orchestrated using Apache Airflow and Azure Data Factory, enabling automated execution of the data workflow. For forecasting, the project uses two different approaches: LSTM (Long Short-Term Memory) and Prophet. Their forecasting performance is evaluated using appropriate evaluation metrics to compare the models. The pipeline also implements watermark-based incremental processing, which allows only newly available data to be processed instead of reprocessing the complete historical dataset.

The proposed architecture demonstrates an end-to-end data engineering and machine learning workflow that is automated, scalable, and suitable for continuous stock-market data processing and forecasting.

1.1 Problem Statement

The continuous generation of stock-market data has created a need for automated and efficient systems that can collect, process, analyze, and forecast financial time-series data. Manual data collection and traditional processing approaches can be time-consuming, require repeated processing of historical data, and make it difficult to maintain an automated forecasting workflow. The problem can be summarized as follows:

- **Manual Data Collection & Processing:** Collecting, cleaning, and processing stock-market data manually is time-consuming and difficult to maintain for regular forecasting activities.
- **Repeated Historical Data Processing:** Processing the complete historical dataset during every pipeline execution results in unnecessary computation and reduces overall pipeline efficiency.
- **Data Quality & Transformation Challenges:** Raw stock-market data may require cleaning, validation, standardization, and transformation before it can be used effectively for analytics and machine learning.
- **Scalability & Pipeline Management:** A reliable data engineering architecture is required to efficiently manage data ingestion, storage, transformation, and machine-learning workflows as the volume of stock-market data increases.
- **Forecasting Accuracy & Model Selection:** Different machine-learning and time-series approaches may produce different forecasting results, making it necessary to compare models such as LSTM and Prophet using suitable evaluation metrics.
- **Lack of End-to-End Automation:** Manual execution of data ingestion, processing, feature engineering, and forecasting makes the overall workflow difficult to manage. An automated cloud-based pipeline is required to integrate these stages into a single end-to-end system.
- **Efficient Incremental Processing:** A mechanism is required to identify and process only newly available data rather than reprocessing the complete historical dataset during every pipeline run.

1.2 Scope

The scope of this project is to design and implement an automated, cloud-based End-to-End Nifty Stock Forecasting Pipeline that integrates data engineering, cloud technologies, and machine learning for efficient stock-market data processing and forecasting. The scope of the project includes the following:

- **Automated Data Collection:** Collect historical Nifty stock-market data from Yahoo Finance using the yfinance library and make it available for further processing.
- **Cloud-Based Data Storage:** Store stock-market data in Azure Data Lake Storage Gen2 using a structured Medallion Architecture.
- **Data Processing & Transformation:** Use Azure Databricks and PySpark to clean, validate, standardize, and transform the collected stock-market data.
- **Medallion Architecture:** Organize data into Bronze, Silver, and Gold layers, where Bronze stores raw data, Silver stores cleaned data, and Gold contains feature-engineered and analytics-ready data.
- **Pipeline Orchestration:** Use Apache Airflow and Azure Data Factory to automate and orchestrate the different stages of the data pipeline.
- **Feature Engineering:** Generate suitable features from historical stock-market data to prepare the dataset for machine-learning-based forecasting.
- **Stock Price Forecasting:** Implement LSTM and Prophet models to forecast future Nifty stock prices using historical time-series data.
- **Model Comparison & Evaluation:** Compare the forecasting performance of LSTM and Prophet using appropriate evaluation metrics and analyze their results.
- **Incremental Processing:** Implement a watermark-based incremental processing mechanism to process only newly available data and avoid unnecessary reprocessing of historical records.
- **Quality Assurance:** Apply data validation, testing, exception handling, and pipeline monitoring practices to improve the reliability of the overall workflow.

1.3 Aim & Objectives

Aim:

The main aim of this project is to design and implement an automated, cloud-based End-to-End Nifty Stock Forecasting Pipeline that integrates data ingestion, processing, storage, feature engineering, and machine learning to generate stock-price forecasts efficiently.

Objectives:

- **Automated Data Ingestion:** To automatically collect Nifty stock-market data from Yahoo Finance using the yfinance library.
- **Cloud-Based Data Storage:** To store and manage stock-market data using Azure Data Lake Storage Gen2.
- **Data Processing:** To clean, validate, standardize, and transform raw stock data using Azure Databricks and PySpark.
- **Medallion Architecture:** To implement Bronze, Silver, and Gold layers for organized and systematic data management.
- **Pipeline Orchestration:** To automate the end-to-end workflow using Apache Airflow and Azure Data Factory.
- **Feature Engineering:** To create relevant features from historical stock data for machine-learning-based forecasting.
- **Forecasting:** To implement LSTM and Prophet models for predicting future Nifty stock prices.
- **Model Comparison:** To compare LSTM and Prophet using suitable evaluation metrics and analyze their forecasting performance.
- **Incremental Processing:** To implement a watermark-based mechanism that processes only newly available data and avoids unnecessary reprocessing.
- **Quality Assurance:** To implement data validation, testing, exception handling, and monitoring practices to improve pipeline reliability.
- **Scalable Architecture:** To develop a production-oriented pipeline that can be extended to handle larger volumes of stock-market data and additional forecasting models.

1.4 Business Problem

Financial markets generate large volumes of stock-market data that need to be collected, processed, and analyzed regularly to support timely financial analysis and forecasting. Manual and non-automated approaches can make this process time-consuming, inefficient, and difficult to scale. The business problem addressed by this project can be summarized as follows:

- **Manual Data Processing:** Financial data teams may spend significant time collecting, cleaning, and preparing stock-market data before it can be used for analysis and forecasting.
- **Delayed Data Availability:** Manual or poorly orchestrated workflows can delay the availability of processed and analytics-ready stock data.
- **High Processing Overhead:** Reprocessing complete historical datasets during every pipeline execution increases computational cost and reduces processing efficiency.
- **Scalability Challenges:** As the volume of historical and newly generated stock-market data increases, the system needs a scalable cloud-based architecture for storage and processing.
- **Difficulty in Forecasting Analysis:** Using only a single forecasting approach may not provide sufficient insight into model performance. Comparing different forecasting techniques helps identify a more suitable model for the given dataset.
- **Lack of End-to-End Automation:** Separate manual steps for data collection, processing, feature engineering, and forecasting make the overall workflow difficult to maintain and monitor.
- **Need for Reliable Data Processing:** Inaccurate, incomplete, or inconsistent data can affect downstream analysis and forecasting results, making data validation and quality checks essential.

The proposed project addresses these challenges by providing an automated cloud-based pipeline that integrates data ingestion, processing, storage, feature engineering, machine learning, model evaluation, and incremental processing into a single end-to-end workflow.

CHAPTER 2
PROJECT DESCRIPTION

This project implements an automated Nifty Stock Forecasting Pipeline designed to collect, process, and forecast stock-market data using cloud-based data engineering and machine learning technologies. The system is built using Microsoft Azure, Apache Airflow, Azure Data Factory, Azure Databricks, and PySpark, following a Medallion Architecture with Bronze, Silver, and Gold layers.

The architecture is designed to be automated, scalable, and efficient, enabling systematic ingestion and processing of Nifty stock-market data from Yahoo Finance, incremental processing using a watermark mechanism, and machine-learning-based forecasting using LSTM and Prophet. The processed data and forecasting outputs are stored in Azure Data Lake Storage Gen2, providing a structured foundation for further analysis, model evaluation, and future enhancements.

2.1 Project Work Flow

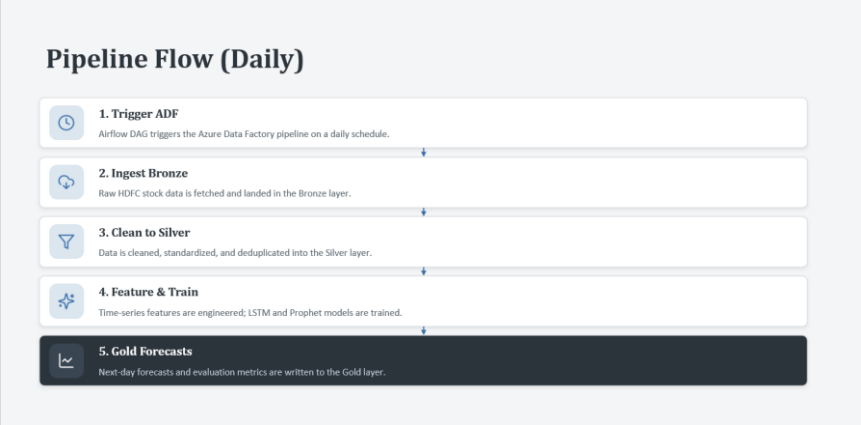

Fig 1: Project work flow diagram

2.2 Project Architecture Overview

The architecture follows an automated, cloud-based data engineering and machine learning model with the following major components:

- **Yahoo Finance & yfinance:** Source for collecting historical Nifty stock-market data containing Date, Open, High, Low, Close, and Volume.
- **Apache Airflow:** Used for workflow scheduling and orchestration, triggering the pipeline automatically.
- **Azure Data Factory (ADF):** Used for Azure-based pipeline orchestration and data movement.
- **Azure Data Lake Storage Gen2 (ADLS Gen2):** Primary cloud storage for maintaining the Bronze, Silver, and Gold data layers.

- **Azure Databricks:** Provides the processing environment for executing notebooks and performing data engineering and machine-learning operations.
- **PySpark:** Used within Databricks for data cleaning, transformation, validation, and feature engineering.
- **Bronze Layer:** Stores the raw Nifty stock-market data with minimal transformation.
- **Silver Layer:** Stores cleaned, standardized, and validated stock-market data.
- **Gold Layer:** Stores feature-engineered and analytics-ready data used for machine-learning forecasting.
- **LSTM & Prophet Models:** Use the Gold-layer data to generate stock-price forecasts and provide a basis for model comparison.
- **Watermark Mechanism:** Tracks the last processed data point and enables incremental processing by processing only newly available data.
- **Forecast Output:** Stores the generated forecasts, engineered features, and model evaluation results for further analysis.

2.3 Data Source & Collection

- **Source:** Yahoo Finance, accessed using the yfinance Python library, is used as the primary source for collecting Nifty stock-market data.
- **Data Collection:** Historical Nifty stock-market data is downloaded programmatically and ingested into the Bronze layer of Azure Data Lake Storage Gen2 (ADLS Gen2).
- Data Attributes:
- **Date:** Trading date of the stock record.
- **Open:** Opening price of the stock for the trading day.
- **High:** Highest price reached during the trading day.
- **Low:** Lowest price reached during the trading day.
- **Close:** Closing price of the stock for the trading day and the primary forecasting target.
- **Volume:** Total number of shares traded during the trading day.
- **Data Storage:** The collected raw data is stored in the Bronze layer of ADLS Gen2 with minimal transformation to preserve the original source data.
- **Data Processing:** The raw data is subsequently processed using Azure Databricks and PySpark for cleaning, validation, standardization, and feature engineering.
- **Incremental Collection:** A watermark-based mechanism is used to identify the last processed data point and process only newly available records during subsequent pipeline executions.

2.4 Dataset Details

The project uses historical Nifty stock-market data collected from Yahoo Finance using the yfinance Python library. The dataset contains daily time-series information that is used for data analysis, feature engineering, and stock-price forecasting.

- **Source:** Yahoo Finance using yfinance
- **Data Type:** Historical daily time-series stock data
- **Columns:** Date, Open, High, Low, Close, and Volume
- **Dataset Size:** The dataset contains 35000+ rows and 6 columns.
- **Target Variable:** Close price, which is used for stock-price forecasting.
- **Data Storage:** Raw data is stored in the Bronze layer of Azure Data Lake Storage Gen2 and then processed through the Silver and Gold layers.
- **Data Processing:** The dataset is cleaned, validated, and transformed using Azure Databricks and PySpark.
- **Purpose:** The processed dataset is used for EDA, feature engineering, LSTM and Prophet model training, forecasting, and model performance evaluation.

2.5 Exploratory Data Analysis (EDA)

Exploratory Data Analysis was performed on the collected Nifty stock-market dataset to understand the structure, quality, distribution, and patterns present in the data before applying machine learning models.

- **Data Structure Analysis:** Checked the number of rows, columns, data types, and overall structure of the dataset.
- **Missing Value Analysis:** Identified and handled missing or null values to ensure data quality.
- **Duplicate Check:** Checked for duplicate records and removed them where required.
- **Statistical Analysis:** Analyzed descriptive statistics such as mean, minimum, maximum, and standard deviation for numerical columns.
- **Price Trend Analysis:** Analyzed the historical movement of Open, High, Low, and Close prices to identify overall market trends and fluctuations.
- **Volume Analysis:** Examined trading volume to understand changes in market activity over time.
- **Correlation Analysis:** Studied the relationship between numerical features such as Open, High, Low, Close, and Volume.
- **Outlier Analysis:** Checked for unusual price or volume values that could affect data processing and model performance.

The EDA findings were used to understand the characteristics of the stock-market data and to determine the appropriate preprocessing and feature-engineering steps before training the LSTM and Prophet forecasting models.

2.6 EDA Findings & Conclusions

- **Data Quality:** Missing values, duplicate records, and data types were checked to ensure the dataset was suitable for further processing.
- **Price Relationship:** Open, High, Low, and Close prices showed strong relationships with each other, reflecting daily stock-price movement.
- **Price Trend:** The Close price showed continuous fluctuations over time, confirming the time-series nature of the dataset.
- **Trading Volume:** Volume varied across trading days and provided additional information about market activity.
- **Feature Relationships:** Correlation analysis helped identify relationships among numerical variables and supported feature selection.
- **Time-Series Pattern:** Historical stock prices showed sequential and time-dependent behavior, making time-series forecasting techniques appropriate.

Conclusion: The EDA confirmed that the dataset was suitable for feature engineering and stock-price forecasting using LSTM and Prophet. The findings from EDA were used to guide the preprocessing and model-development stages of the pipeline.

CHAPTER 3
SYSTEM ARCHITECTURE

3.1 Technologies Used & Their Roles

The project combines cloud services, data engineering tools, workflow orchestration, and machine learning technologies to build an automated stock forecasting pipeline.

- **Python & yfinance:** Python is used for data collection and processing, while yfinance is used to fetch historical Nifty stock-market data from Yahoo Finance.
- **Apache Airflow:** Used to schedule and automate the execution of the pipeline workflow.
- **Azure Data Factory:** Used for data movement and orchestration within the Azure environment.
- **Azure Data Lake Storage Gen2:** Used as the central storage system for maintaining the Bronze, Silver, and Gold layers.
- **Azure Databricks & PySpark:** Databricks provides the processing environment, while PySpark is used for data cleaning, transformation, validation, and feature engineering.
- **LSTM:** Used as a deep-learning forecasting model to learn sequential patterns from historical stock-price data.
- **Prophet:** Used as a time-series forecasting model to capture trends and provide a comparison with LSTM.
- **Watermark Mechanism:** Used to track the last processed data point and enable incremental processing, reducing unnecessary reprocessing of historical data.

3.2 Azure Services & Their Roles

The project uses Microsoft Azure services to provide scalable cloud storage, data processing, and pipeline management.

- **Azure Data Lake Storage Gen2 (ADLS Gen2):** Stores raw, cleaned, and processed stock-market data using Bronze, Silver, and Gold layers.
- **Azure Data Factory (ADF):** Manages data movement and helps orchestrate the pipeline workflow.
- **Azure Databricks:** Provides a scalable environment for PySpark-based data processing, feature engineering, and machine-learning operations.

These services provide centralized storage, scalable processing, and integration between different stages of the pipeline.

3.3 Apache Airflow & Azure Data Factory Orchestration

Apache Airflow is used as the workflow orchestration tool to schedule and automate the execution of the pipeline. The Airflow DAG manages the sequence of pipeline activities and triggers the required workflow.

Azure Data Factory (ADF) is used for data movement and Azure pipeline orchestration. Together, Airflow and ADF help automate the flow from data ingestion to processing and forecasting, reducing manual intervention.

3.4 Azure Databricks & PySpark Processing

Azure Databricks is used as the main processing environment for executing the project's notebooks. It provides a scalable platform for processing stock-market data and performing machine-learning operations.

PySpark is used within Databricks for distributed data processing. It performs operations such as data cleaning, validation, transformation, standardization, and feature engineering before the data is used for forecasting.

3.5 Medallion Architecture (Bronze, Silver & Gold Layers)

The project follows the Medallion Architecture, which organizes data into three logical layers: Bronze, Silver, and Gold.

- **Bronze:** Stores raw data as received from the source. Raw Nifty stock-market data collected using yfinance is stored with minimal transformation, preserving the original data for future processing and validation.
- **Silver:** Contains cleaned, validated, and standardized data. Bronze data is cleaned using PySpark, and missing values, duplicates, data types, and data-quality issues are handled before storing the processed data.
- **Gold:** Contains feature-engineered and analytics-ready data. Feature engineering is performed on the Silver data to create forecasting-related features used by the LSTM and Prophet models.

This layered approach improves data organization, quality, traceability, and maintainability.

3.6 Advantages, Limitations & Technology Selection

Technology Selection

The technologies were selected based on their suitability for building an automated, scalable, and cloud-based data engineering and machine-learning pipeline.

- **Azure:** Selected for cloud-based storage, processing, and integration.
- **Databricks & PySpark:** Selected for scalable data processing and transformation.
- **Airflow & ADF:** Selected to automate and orchestrate different pipeline activities.
- **LSTM:** Selected because it is suitable for learning sequential patterns in time-series data.
- **Prophet:** Selected because it provides a simpler and interpretable time-series forecasting approach for comparison.

Advantages

- Automated and repeatable pipeline execution.
- Scalable cloud-based data storage and processing.
- Organized data management using Medallion Architecture.
- Incremental processing reduces unnecessary computation.
- Comparison of LSTM and Prophet provides multiple forecasting approaches.

Limitations

- Stock prices are highly volatile and difficult to predict accurately.
- Forecasting performance depends on the quality and availability of historical data.
- LSTM requires more computational resources and training time compared with simpler forecasting models.
- External factors such as news, market sentiment, and economic events are not fully captured by historical price data.

CHAPTER 4
IMPLEMENTATION DETAILS

4.1 Data Preprocessing & Feature Engineering

The raw stock-market data collected from Yahoo Finance is processed using PySpark in Azure Databricks before being used for forecasting.

- **Data Cleaning:** Missing values, duplicate records, and invalid data types are checked and handled.
- **Data Transformation:** Date and numerical columns are converted into appropriate formats for further processing.
- **Data Validation:** The processed data is checked for consistency and data-quality issues.
- **Feature Engineering:** Relevant time-series features such as daily returns, moving averages, volatility, and lag values are generated from historical stock prices.
- **Data Preparation:** The processed features are organized into a suitable format for training the LSTM and Prophet models.

4.2 LSTM Model Training & Forecasting

Long Short-Term Memory (LSTM) is a recurrent neural network designed to work effectively with sequential and time-series data.

- Historical stock-price data is prepared as sequential input for the model.
- The data is divided into training and testing datasets.
- The LSTM model learns patterns and dependencies from historical stock-price sequences.
- The trained model is used to predict future stock prices.
- The predicted values are compared with actual values to evaluate forecasting performance.

LSTM is useful for this project because stock prices contain sequential patterns and dependencies across different time periods.

4.3 Prophet Model Training & Forecasting

Prophet is a time-series forecasting model that can capture trends and seasonal patterns in historical data.

- The dataset is prepared in the required ds and y format.
- Historical Close-price data is used for model training.
- Prophet learns the underlying trend and time-based patterns.
- The trained model generates future stock-price forecasts.
- The predictions are evaluated against actual values using forecasting metrics.

Prophet provides a simpler and more interpretable forecasting approach and acts as a comparative model against LSTM.

4.4 LSTM vs Prophet Model Comparison

The project uses two different forecasting approaches to compare their performance.

Feature	LSTM	Prophet
Type	Deep Learning	Time-Series Forecasting
Main Strength	Sequential pattern learning	Trend and seasonality
Training Complexity	Higher	Lower
Interpretability	Lower	Higher
Training Time	Generally higher	Generally faster
Use in Project	Primary deep-learning approach	Comparative forecasting approach

The models are compared using the same evaluation criteria to determine which approach performs better on the selected stock-market dataset.

4.5 Model Evaluation Metrics

The forecasting models are evaluated by comparing predicted prices with actual stock prices. Suitable metrics such as MAE (Mean Absolute Error), RMSE (Root Mean Squared Error), and MAPE (Mean Absolute Percentage Error) can be used for performance evaluation.

- **MAE:** Measures the average absolute difference between actual and predicted values.
- **RMSE:** Gives greater importance to larger prediction errors.
- **MAPE:** Represents prediction error as a percentage of actual values.

The model with lower error values is considered to have better forecasting performance for the selected evaluation period.

4.6 Incremental Processing & Watermark Strategy

To avoid repeatedly processing the complete historical dataset, the project implements a watermark-based incremental processing strategy.

- A watermark records the last successfully processed data point.

- During the next pipeline execution, the watermark is checked.
- Only newly available data after the last processed point is selected.
- The new data is processed and added to the appropriate data layer.
- The watermark is updated after successful processing.

This approach reduces unnecessary computation, improves pipeline execution time, and makes the workflow more production-oriented.

4.7 Code Implementation & Explanation

The project is implemented using Python, PySpark, and Databricks notebooks. The implementation is divided into separate stages to maintain a clear and modular workflow.

- **Notebook 1:** Handles stock-data collection and ingestion into the Bronze layer.
- **Notebook 2:** Reads Bronze data, performs cleaning and validation, and writes the processed data to the Silver layer.
- **Notebook 3:** Performs feature engineering and prepares the Gold-layer dataset for forecasting.
- **ML Implementation:** LSTM and Prophet models are trained using the prepared data and their predictions are evaluated.
- **Pipeline Parameters:** Configurable parameters and widgets are used where required to make the notebooks reusable.

The code is organized into logical stages so that each part of the pipeline can be tested, maintained, and explained independently.

4.8 Problems Faced & Solutions

During implementation, several technical challenges were encountered and resolved.

- **Data Ingestion Issues:** Problems related to collecting and storing stock data were handled by validating the downloaded dataset before ingestion.
- **Data Quality Issues:** Missing values, duplicates, and inconsistent data types were identified during preprocessing and handled using PySpark transformations.
- **Repeated Processing:** Reprocessing the complete historical dataset was inefficient. A watermark mechanism was implemented for incremental processing.
- **Large-Scale Processing:** PySpark was used in Databricks to perform scalable data transformations.
- **Model Training Challenges:** Stock prices are highly variable and sequential. Data preprocessing, feature engineering, and proper train-test preparation were used before model training.
- **Pipeline Execution Issues:** Pipeline stages were separated into notebooks and orchestrated using Airflow and Azure Data Factory to improve workflow management.

4.9 Quality Assurance, Testing & Exception Handling

Quality assurance practices are applied throughout the pipeline to improve reliability and ensure that incorrect data does not move to downstream stages.

- **Data Validation:** Checks are performed for null values, duplicate records, data types, and required columns.
- **Schema Validation:** Expected columns and data formats are verified before processing.
- **Pipeline Testing:** Individual notebooks and pipeline stages are tested separately before complete pipeline execution.
- **Exception Handling:** Python and PySpark exception-handling mechanisms are used to manage runtime and data-processing errors.
- **Incremental Processing Validation:** The watermark is checked to ensure that already processed data is not unnecessarily processed again.
- **Model Validation:** Forecast results are compared with actual values using evaluation metrics.
- **Output Validation:** Final processed data and forecasting outputs are checked before being considered successful.

CHAPTER 5
PROJECT REQUIREMENTS & RESULTS

5.1 Hardware Requirements

The project requires a system capable of supporting cloud-based development, data processing, and machine-learning activities.

- **Processor:** Intel Core i5 or equivalent
- **RAM:** Minimum 8 GB recommended
- **Storage:** Minimum 256 GB available storage
- **Internet:** Stable internet connection for accessing Azure services and Yahoo Finance
- **Cloud Resources:** Azure resources are used for scalable storage and processing.

5.2 Software & Libraries Used

The following software, frameworks, and libraries are used in the project:

- **Python:** Data collection, preprocessing, and machine learning
- **PySpark:** Distributed data processing and transformation
- **yfinance:** Stock-market data collection
- **TensorFlow / Keras:** LSTM model development and training
- **Prophet:** Time-series forecasting
- **Apache Airflow:** Workflow scheduling and orchestration
- **Azure Data Factory:** Data movement and pipeline orchestration
- **Azure Databricks:** Data processing and notebook execution
- **Azure Data Lake Storage Gen2:** Cloud-based data storage

5.3 Azure Resources & Configuration

The project uses the following Azure resources:

- **Azure Data Lake Storage Gen2:** Stores the Bronze, Silver, and Gold data layers.
- **Azure Data Factory:** Used to manage data movement and pipeline activities.
- **Azure Databricks:** Used for PySpark processing, feature engineering, and machine-learning operations.
- **Storage Configuration:** Separate paths are maintained for raw, processed, and analytics-ready data.

The Azure resources are configured to support secure, organized, and scalable processing of stock-market data.

5.4 Pipeline Deployment & Execution

The pipeline is deployed as an automated workflow using Apache Airflow, Azure Data Factory, and Azure Databricks.

- Airflow schedules and triggers the pipeline workflow.
- Azure Data Factory manages the required data movement and orchestration activities.
- Databricks notebooks perform data ingestion, cleaning, transformation, and feature engineering.
- LSTM and Prophet models generate stock-price forecasts using the Gold-layer data.
- The final processed data, forecasts, and evaluation results are maintained in the appropriate Azure storage locations.

The pipeline can be executed repeatedly while using watermark-based incremental processing to avoid unnecessary reprocessing of historical data.

5.5 Forecasting Results

The forecasting stage generates predicted stock prices using both LSTM and Prophet models.

- Historical data is used for model training.
- The trained models generate forecasts for the selected future period.
- Predicted values are compared with actual stock prices.
- Evaluation metrics are used to measure forecasting errors.

The final results help determine how effectively each model performs on the selected stock-market dataset.

5.6 Model Performance Comparison

The performance of LSTM and Prophet is compared using forecasting evaluation metrics such as MAE, RMSE, and MAPE.

Model	Approach	Performance
LSTM	Deep Learning / Sequential Model	Based on actual evaluation results
Prophet	Time-Series Forecasting	Based on actual evaluation results

The model with lower prediction-error values is considered better for the selected dataset and evaluation period. Actual metric values should be added from the project's model output rather than using assumed values.

5.7 Project Screenshots

The following screenshots can be included to demonstrate the implementation and deployment of the project:

- Yahoo Finance / data collection output

- Azure Data Lake Storage Bronze, Silver, and Gold folders
- Azure Data Factory pipeline
- Apache Airflow DAG and execution status
- Azure Databricks notebooks
- Data processing results
- Feature-engineered Gold dataset
- LSTM and Prophet forecasting results
- Model evaluation metrics
- Final pipeline execution / output

CHAPTER 6
FUTURE SCOPE

- **Additional Data Coverage:** Adding more Nifty stocks and longer historical datasets to broaden the forecasting base.
- **Sentiment Integration:** Incorporating news sentiment and other external market indicators to improve forecasting accuracy.
- **Additional Models:** Adding further forecasting and machine-learning models for broader comparison.
- **Automated Retraining:** Implementing automated pipelines to retrain models regularly as new data becomes available.
- **Monitoring & Alerting:** Adding monitoring and alerting for pipeline failures and data-quality issues.
- **Interactive Dashboards:** Developing a dashboard using Power BI for interactive visualization of forecasts and model performance.
- **Real-Time Deployment:** Deploying the forecasting model as a real-time prediction service.

CHAPTER 7
CONCLUSION

This project successfully implements an End-to-End Nifty Stock Forecasting Pipeline by integrating cloud-based data engineering, automated orchestration, distributed data processing, and machine learning. The pipeline efficiently collects historical stock-market data from Yahoo Finance and processes it through Azure Data Lake Storage Gen2, Azure Databricks, PySpark, Apache Airflow, and Azure Data Factory.

The implementation of Medallion Architecture organizes the data into Bronze, Silver, and Gold layers, ensuring systematic data storage, cleaning, transformation, and feature engineering. The use of watermark-based incremental processing reduces unnecessary reprocessing of historical data and improves overall pipeline efficiency.

For stock-price forecasting, LSTM and Prophet models are implemented and evaluated using appropriate forecasting metrics. The comparison of both models provides insights into their performance on the selected stock-market dataset. Data validation, testing, exception handling, and pipeline-level quality checks are also incorporated to improve the reliability of the solution.

The major learnings from the project include designing an end-to-end cloud-based data pipeline, implementing Bronze, Silver, and Gold layers using Medallion Architecture, performing distributed data processing using PySpark, automating workflows using Airflow and Azure Data Factory, implementing incremental processing using a watermark mechanism, applying LSTM and Prophet to time-series forecasting, performing model comparison and evaluation, and implementing data validation, testing, and exception handling.

The project also has certain limitations: stock prices are highly volatile and difficult to predict accurately, historical price data alone cannot capture all external market factors such as news, investor sentiment, and economic conditions, LSTM training can require more computational resources and tuning, and forecasting performance may vary depending on the stock and time period selected.

Overall, this project demonstrates a scalable, automated, and production-oriented approach to stock-market data engineering and forecasting. Future improvements can include incorporating additional market indicators and news sentiment, adding more forecasting models, implementing automated model retraining, and developing real-time dashboards for monitoring forecasts and pipeline performance.

CHAPTER 8
REFERENCES

1. Python Software Foundation. Python Documentation. https://docs.python.org/3/

2. yfinance Documentation. https://pypi.org/project/yfinance/

3. Apache Airflow Documentation. https://airflow.apache.org/docs/

4. Microsoft Azure Documentation: Azure Data Factory. https://learn.microsoft.com/en-us/azure/data-factory/

5. Microsoft Azure Documentation: Azure Data Lake Storage Gen2. https://learn.microsoft.com/en-us/azure/storage/blobs/data-lake-storage-introduction

6. Microsoft Azure Documentation: Azure Databricks. https://learn.microsoft.com/en-us/azure/databricks/

7. Apache Spark Documentation: PySpark API. https://spark.apache.org/docs/latest/api/python/

8. Pandas Documentation. https://pandas.pydata.org/docs/

9. TensorFlow/Keras Documentation: LSTM Layers. https://www.tensorflow.org/api_docs/python/tf/keras/layers/LSTM

10. Prophet Documentation (Meta/Facebook Core Data Science Team). https://facebook.github.io/prophet/

11. Scikit-learn Documentation: Model Evaluation Metrics. https://scikit-learn.org/stable/modules/model_evaluation.html